

SI-Mapper: Agentic AI for Automated HVAC Semantic Modeling

From BACnet Data and Engineering Drawings to ASHRAE 223P Ontologies

Juan [Last Name]
[Affiliation]

March 2026

Abstract

This report presents SI-Mapper, an agentic AI system that automates the creation of ASHRAE 223P semantic building models from HVAC engineering drawings and BACnet point data. We describe the problem of manual BACnet point mapping — a costly, labor-intensive process that blocks building software interoperability and grid integration — and present an AI-driven solution that replaces weeks of expert engineering work. The report traces the development through nine experimental iterations, from simple function-calling approaches to the final skills-based single master agent architecture, and provides a framework for comparing AI-generated results against human engineer baselines.

Contents

1	Introduction and Context	3
1.1	Building Ontologies: A Standardized Language for Buildings	3
1.2	ASHRAE Standard 223P	4
1.3	Advantages and Barriers of Semantic Building Models	4
2	Description of the Problem	6
2.1	The BACnet Point Mapping Problem	6
2.2	The Cost of Manual Mapping	6
2.3	The Research Gap and Emerging Solutions	7
2.4	Why This Matters for the Energy Grid	9
3	SI-Mapper: An Agentic Approach to Building Semantic Modeling	10
3.1	Overview of the Approach	10
3.2	Development Methodology: Nine Experimental Iterations	11
3.3	Final Architecture	13
3.4	Skills	14
3.5	Tools	14
3.6	Key Technology Decisions	14
4	Experiment description, analysis, and results	16
4.1	Experimental Setup	16
4.2	System 1: Air Handling Unit	17
4.2.1	Input Materials	17
4.2.2	Performance Metrics	17
4.2.3	Graph Quality Assessment	18
4.3	System 3: Air Handling Unit	18
4.4	Human Engineer Baseline	19

4.5	Comparison: AI Agent vs. Human Engineer	19
5	Conclusions	19
5.1	Summary of Contributions	19
5.2	Limitations	20
5.3	Future Work	21

1 Introduction and Context

Every commercial building contains hundreds of sensors, actuators, and control points. A modern office block may have thousands of temperature sensors, pressure transducers, damper position indicators, and setpoint registers, each one a source of valuable operational data. Yet despite this wealth of instrumentation, the data is largely inaccessible to software that was not purpose-built for the specific building it operates in, i.e., software cannot be easily ported from one building to another.

The reason is a fundamental naming problem. Each Building Management System (BMS) vendor uses its own proprietary conventions for labelling control points. A temperature sensor named 2500.AI11 in one system becomes RTU-3/SA-T in another. Neither label carries enough information for software to determine automatically what physical equipment the point belongs to, what role the measurement plays, or how that equipment connects to other components. Without that context, building software cannot be deployed without a building-specific customization step. This means that “building software is not ‘plug and play’, technicians must manually map sensors and actuators to software inputs through tedious point mapping processes that vary building-by-building.” Moreover, this manual mapping, a building-by-building effort, is error-prone and expensive, often requiring weeks of expert labor for a single building.

A promising solution for this problem is a *standardized metadata layer*, a machine-readable description of what every sensor and actuator means, which equipment it belongs to, and how that equipment connects to the rest of the system. This is exactly what building ontologies provide.

1.1 Building Ontologies: A Standardized Language for Buildings

A building ontology is a shared, machine-readable vocabulary that describes building systems, their components, and how they connect between each other. Think of it as a dictionary that any software application can consult: if a sensor is declared to be a supply air temperature measurement on the outlet of a fan coil unit, any compliant application (that understands the chosen ontology) can understand that without manual configuration.

Four major building ontologies have emerged over the past decade. Each reflects different design priorities and is backed by a different community.

Table 1: The four major building ontology standards.

Ontology	Governed By	Approach	Focus
Project Haystack	Haystack Connect	Tag-based dictionary	Equipment types, sensor tags
Brick Schema	Open consortium	RDF/OWL semantic graph	Equipment, sensors, spatial hierarchy
RealEstateCore	RealEstateCore Consortium	OWL ontology	Property management + technical sys
ASHRAE 223P	ASHRAE + DOE/NREL	Formal OWL ontology	Topology, connections, telemetry

Project Haystack is the oldest and most widely deployed approach. It uses a tag-based system where sensors are described by a set of keywords (e.g., **air temp sensor**). Its informality makes it easy to adopt but difficult to enforce consistently across projects.

Brick Schema builds formal semantic rules on top of a Haystack-compatible vocabulary. It uses the RDF/OWL knowledge representation standard, making it more consistent and machine-interpretable than Haystack. Brick is production-ready and in active deployment across research institutions and building operators.

RealEstateCore focuses on the property management perspective, occupancy, asset management, and energy performance, with significant overlap into technical building systems. It is used mainly in the European commercial real estate sector.

ASHRAE 223P is the most rigorous of the four. It defines not just equipment types but full system topology: how equipment connects, what flows between components, and how those connections link to live sensor data. ASHRAE’s BACnet committee, Project Haystack, and Brick Schema are actively collaborating to integrate their vocabularies into ASHRAE 223P as the unifying standard.

Nevertheless, no single standard covers all needs that can arise in practice when modeling complex building systems. A complete building model is expected to be “a mix of ASHRAE 223P, Brick, QUDT, RealEstateCore, and possibly additional standards.” ASHRAE 223P is designed as the semantic backbone that ties the others together.

1.2 ASHRAE Standard 223P

ASHRAE Standard 223P is a proposed ASHRAE standard that formally defines a knowledge representation framework for building systems. Its goal is to enable software applications to determine the meaning and context of building data without manual, building-specific configuration.

Current status: The standard is in its second advisory public review as of 2025-2026. The project receives funding from the Department of Energy and is led by the National Renewable Energy Laboratory (NREL), with participation from Lawrence Berkeley National Lab, Pacific Northwest National Lab, NIST, Colorado School of Mines, and Cornell University.

ASHRAE 223P organizes building system information around six core concepts, described in Table 2.

Table 2: The six core concepts of ASHRAE 223P.

Concept	Definition	Example
Type	Well-defined classes with formal rules governing usage	Fan, Pump, CoolingCoil
Topology	How equipment and spaces connect	Fan outlet → Duct → Coil inlet
Composition	Hierarchical grouping of entities	AHU contains Fan, Coil, Damper
Telemetry	Observable properties with external data references	Supply air temp → BACnet object 2500.AI11
Characteristics	Descriptive attributes with units	Rated airflow in m ³ /s
Medium	Substance conveyed through connections	Chilled water, Supply air, Electricity

Topology is the most distinctive concept in ASHRAE 223P. Where other ontologies describe what equipment exists, 223P also describes how equipment connects. Every connection has a direction (inlet, outlet, or bidirectional), a physical path (duct, pipe, or wire), and a medium (air, water, electricity). This means an ASHRAE 223P model captures not just “there is a fan and a coil” but “air flows from the fan outlet through a duct segment into the coil inlet, conveying Supply Air at design conditions.”

Telemetry and external references allow live BACnet sensor data to be explicitly linked to semantic properties in the model. A supply air temperature sensor in the model can declare that its value comes from BACnet object 2500.AI11, closing the loop between the abstract ontology and the live building management system.

1.3 Advantages and Barriers of Semantic Building Models

Creating a semantic building model is not a trivial investment. Understanding why organizations would make that investment, and why they currently do not, frames the business case for automation tools to build the ontology representation of a building.

Advantages of semantic building models:

- **Software portability:** applications built on a standard ontology can work across buildings without building-specific customization, just as web browsers work across websites.
- **Advanced analytics:** fault detection, automated commissioning, and predictive maintenance all require knowing not just sensor values but what those values mean and how equipment relates to each other.
- **Grid services enablement:** Maximizing the contribution of the buildings to the grid services, it is required a deep understanding of the building composition, building mechanics, and owner preferences. A way to facilitate the extraction of that information for Virtual Power Plants or demand aggregators is by using standard ontologies at the building level.
- **Reduced integration costs:** once a semantic model exists, any compliant application can consume it without a custom integration project.
- **Digital twin support:** the semantic layer is the connective tissue between physical building systems and their digital representations.

Barriers to adoption:

- **High creation cost:** building a semantic model today requires specialized domain expertise and significant labor, often weeks of work by a qualified HVAC engineer.
- **No automated tooling for existing buildings:** most available approaches are manual or semi-manual, requiring expert knowledge of both HVAC systems and ontology standards.
- **Fragmented standards landscape:** organizations must choose between partially overlapping standards with no single dominant solution.
- **Model maintenance:** models drift from reality as building systems are modified, requiring ongoing expert effort to keep them current.
- **Low industry adoption:** despite clear benefits, the broader industry has been slow to adopt formal ontologies, creating a chicken-and-egg problem for software vendors.

The high cost of *creating* semantic models is the central barrier for adoption. A semantic model that would take a qualified engineer several weeks to create manually could enable years of automated analytics and facilitate control films to enable grid integration for the building. The investment is worthwhile; the bottleneck is the creation step. The creation step is, therefore, the central bottleneck blocking wider adoption of semantic building models and the benefits they enable.

This work explores whether recent advances in artificial intelligence can meaningfully reduce that bottleneck. Modern large language models have demonstrated the ability to process heterogeneous, multi-modal data, including equipment schedules, control diagrams, point lists, and floor plans, and to reason over structured knowledge. **SI-Mapper** is a prototype tool that investigates this possibility: given the multi-modal data typically available in a real building project, can an AI-assisted pipeline generate an ASHRAE 223P-compliant ontology with sufficient accuracy and completeness to be practically useful?

ASHRAE 223P is the target standard because it is the only building ontology that simultaneously captures full HVAC topology, links live BACnet telemetry to semantic equipment properties, and is explicitly designed for advanced building controls and grid-services applications. Its formal structure and institutional backing by DOE, NREL, and ASHRAE make it the most credible long-term foundation for interoperability, and its richness makes it the most demanding and informative test bed for evaluating AI-generated building models.

2 Description of the Problem

Section 1 established that building semantic models unlock significant value but are costly to create. This section examines the cost problem in concrete terms. We describe what BACnet points are, quantify the labor required to map them, identify the research gap that has prevented an automated solution, and explain why the same problem blocks the emerging Virtual Power Plant sector.

2.1 The BACnet Point Mapping Problem

A Building Automation System (BAS) exposes operational data through a hierarchy of *points*, individual sensors, actuators, setpoints, alarms, and status flags. In BACnet (the dominant open protocol for commercial building controls), each point is a typed object (Analog Inputs, Binary Outputs, Analog Values, etc.) with a unique address on the network. A typical commercial office building has between 500 and 5,000 BACnet points; a large hospital or campus facility may have tens of thousands [1].

The *mapping problem* is this: a raw list of BACnet points contains no standard information about what physical equipment each point belongs to, what role it plays in the system, or how that equipment connects to other components. A point named AHU-1.SA-T tells a human engineer “probably supply air temperature on Air Handling Unit 1,” but only if the engineer already knows the naming convention used by that particular BMS installer. It is common to find conventions that differ between contractors, between projects, and even between systems within the same building.

Reconstructing the full picture of the building, which points belong to which equipment, what each point means, and how the equipment connects into a complete HVAC system, requires reading as-built HVAC drawings, interrogating BMS programming, and applying expert engineering judgment. This process is manual, time-consuming, and dependent on domain expertise.

Modbus, an even older and simpler protocol used in legacy HVAC controllers, meters, and industrial equipment, exposes data as bare numbered registers with no descriptive metadata at all, making the mapping challenge even more difficult than BACnet.

2.2 The Cost of Manual Mapping

Manual BACnet point mapping and BAS commissioning are widely documented as major cost drivers in building construction and retrofit projects. Table 3 summarizes the key figures from industry research and published case studies.

To illustrate the compounding nature of these costs: the 150,000 sq ft school in the case study paid \$225,000 for commissioning up front, then incurred approximately \$175,000 in unplanned first-year expenses due to inadequate point documentation, nearly doubling the effective cost of the commissioning phase [3]. According to NREL [1], complete BAS installation averages \$7.76 per square foot across the United States, with the legacy systems integration premium adding a further 20 to 40 percent on retrofit projects [4] (data from 2022 and 2023).

Manual approaches are also incomplete by design. Industry practice typically tests only 10 to 20 percent of terminal units during commissioning [3]; every 10 percent reduction in testing coverage correlates to approximately a 6 to 8 percent increase in post-occupancy issues. The result is a pervasive, chronic problem: buildings begin their operational life with under-documented, partially verified control systems, and the cost of correction grows over time. Wang et al. [6] describe manual point mapping as “labor intensive and costly, presenting a major impediment to innovations in building control”, validating the scale of the problem even where precise figures are contractor-dependent.

Table 3: Key cost and effort data for manual BAS commissioning and point mapping.

Cost Item	Figures	Source
BAS commissioning as % of construction	0.5%–1.5% of total; 8%–15% of BAS project value	[2]
Case study: 150,000 sq ft school	\$225,000 commissioning (\$1.50/sq ft)	[3]
Complete BAS installation average	\$7.76/sq ft (range: \$5.20–\$14.18)	[1]
Legacy BMS integration premium	+20%–40% over base cost	[4]
Additional service calls from poor mapping	12–15 extra calls/year; +30%–45% troubleshooting time	[3]
Energy performance drift from poor commissioning	15%–30% energy waste in first 3 years; \$0.20–\$0.50/sq ft/year	[3]
First-year unplanned expenses (case study school)	\$175,000, nearly equal to original commissioning cost	[3]
HVAC technician shortage (2024–2025)	110,000 unfilled positions	[5]

2.3 The Research Gap and Emerging Solutions

Despite the scale and economic impact of the manual mapping problem, no widely-deployed end-to-end automated solution existed as of Wang et al.’s 2017 systematic review [6], which found the field lacked:

- a complete problem formulation with integrated solution approaches,
- standardized test datasets and performance metrics, and
- methods well-aligned with real-world mapping requirements.

The AI wave of 2023 onwards has catalyzed a new generation of attempts to close this gap. Table 4 surveys the most relevant commercial and academic efforts and positions SI-Mapper within the landscape. The Building Commissioning Association’s published guidance [2] confirms that point-to-point verification remains a mandated step on every new BAS project, underscoring that the problem persists even as partial solutions emerge.

The table reveals a clear structural gap. Existing solutions either stop short of semantic modeling (PingCx automates testing but produces no ontology), require clean inputs that do not exist in practice (BrickLLM needs accurate text; BIM2RDF needs a complete BIM file), or are expert-facing development toolkits rather than turnkey automation (BuildingMOTIF). Commercial platforms such as Willow approach the problem from the opposite direction, top-down, starting from live data in large portfolios, using a proprietary ontology that is not aligned with ASHRAE 223P or the emerging interoperability standards required for Virtual Power Plants integration.

National laboratory programs (NREL, LBNL, PNNL, NIST) have made significant progress in defining the 223P standard and associated tooling through BuildingMOTIF [8, 12], but the explicit DOE framing acknowledges the goal is still to “lower costs and reduce installation

Table 4: Competitive landscape for automated BAS point mapping and semantic model generation (2024–2026).

Solution	Category	Input	Gap vs. SI-Mapper
PingCx [3]	Automated functional commissioning & point-to-point testing	Live BACnet connection	Tests whether points respond correctly; does <i>not</i> build a semantic model or identify equipment topology
BrickLLM [7]	AI-driven Brick ontology generation	Free-text building description	Requires accurate natural-language input; does not parse BACnet CSV exports, HVAC drawings, or produce 223P
BuildingMOTIF [8]	DOE open-source SDK for 223P / Brick / Haystack model creation and validation	Expert-authored RDF; BACnet point lists (manual)	Developer toolkit, not an AI inference engine; requires domain expert to drive model creation
BIM2RDF Open223 [9]	/ BIM-to-223P conversion	Revit / IFC BIM file	Requires a complete, accurate BIM as input; does not handle buildings where only as-built drawings and BACnet CSVs exist
Willow [10]	AI digital twin platform with ML device classification	Live BMS / IoT feeds	Uses proprietary DTDL ontology, not 223P; requires live data connection and human-guided onboarding; \$110M-funded commercial platform
SkySpark / Project Haystack [11]	Analytics platform with semantic tagging	Engineer-applied Haystack tags	Tagging is manual; provides analytics engine once tagged, not an automated mapping tool
SI-Mapper (this work)	End-to-end AI pipeline for 223P model generation	BACnet CSV point exports + as-built HVAC drawings	No comparable system addresses the full pipeline from raw BACnet exports and engineering drawings to a validated ASHRAE 223P graph

time”, an admission that the current tooling does not yet achieve this without substantial expert involvement.

The gap SI-Mapper addresses is therefore specific and narrow: *automated end-to-end generation of ASHRAE 223P semantic models from the artifacts that actually exist on the ground*, BACnet CSV exports and as-built engineering drawings, without requiring a pre-existing BIM file, clean natural-language descriptions, or an expert semanticist.

2.4 Why This Matters for the Energy Grid

The point mapping problem is not only a building operations challenge. It is a direct barrier to the energy transition.

Non-Wire Alternatives and **Virtual Power Plants** aggregate distributed energy resources, including commercial building HVAC loads, to provide grid services equivalent to building new power plants or transmission lines. HVAC systems consume approximately 40 percent of total commercial building energy and can be modulated to respond to grid signals, shifting load away from peak demand periods. In aggregate, this flexibility represents enormous potential grid capacity.

The Virtual Power Plants market is growing rapidly. North American Virtual Power Plants programs totaled 37.5 GW of flexible behind-the-meter capacity as of 2025, up approximately 14 percent from 2024, with active deployments rising more than 33 percent in the same period [13]. Policy support is accelerating: ten state legislatures introduced Virtual Power Plants bills in 2024 [14, 15]; Massachusetts utilities are implementing a \$50 million Grid Services Compensation Fund to promote non-wire alternatives [14]; and the California Energy Commission awarded a \$2 million EPRI grant specifically to demonstrate Virtual Power Plants for commercial buildings including HVAC controls [16].

How aggregators operate today. Two types of aggregators participate in these programs. *Hardware aggregators* manufacture controllable devices (smart thermostats, batteries, EV chargers) and aggregate their own installed base into a dispatchable resource portfolio. *Software aggregators* act as protocol proxies, connecting heterogeneous devices from multiple vendors under a single management layer. Both types sell capacity and demand response services to utilities and grid operators, who dispatch them using standardized signals such as OpenADR or IEEE 2030.5. This model works efficiently for residential assets and fleets of identical devices because the aggregator controls the endpoint hardware and already knows its capabilities.

Commercial buildings are structurally different. A commercial building’s HVAC system is not a device the aggregator manufactured. It was designed by an engineering firm, installed by a mechanical contractor, and programmed by a BMS provider (Siemens, Johnson Controls, ABB, Niagara, or equivalent). Each of these parties has a legitimate interest in protecting the reliability of the system: the BMS provider guarantees uptime; the commissioning engineer designed the control sequences; the building operator is legally responsible for occupant comfort. None of them will willingly allow an external party to write arbitrary setpoints to their system without a formal agreement and a clear technical boundary. The result is that every commercial building enrollment today requires a custom, building-by-building negotiation and integration project [17]. The DOE Virtual Power Plants Liftoff report identifies that no industry standardization exists. Virtual Power Plants vendors have been building their solutions from the ground up, then working on a case-by-case basis with utilities to develop customized solutions, resulting in rigid systems that can’t easily be replicated in new areas [17].

How an ASHRAE 223P ontology changes the logistics. An ASHRAE 223P semantic model of a building is not a control system replacement, it is a machine-readable contract that describes what is controllable, how to reach it, and under what conditions. Consider the operational chain it enables:

1. **Asset discovery.** The ontology declares every controllable component (fans, coils, VAV

boxes, dampers) as a typed entity with its position in the HVAC topology. An aggregator can query it to answer “what loads does this building contain and what are their nominal capacities?” without a site visit or manual documentation review.

2. **Live telemetry access.** ASHRAE 223P’s *external reference* mechanism links each semantic property directly to its BACnet object address. A supply air temperature sensor in the model carries the address of the physical BACnet point that reports its value. The aggregator reads live sensor data by following those references, with no custom integration code per building.
3. **Flexibility estimation.** With topology and live telemetry available, an algorithm can compute how much load the building can shed, for how long, and with what thermal lag, before comfort constraints are violated. This estimate feeds directly into the aggregator’s dispatch optimization.
4. **Controlled dispatch.** The ontology also identifies the BACnet addresses of controllable setpoints (supply air temperature setpoint, fan speed command, cooling setpoint). When the grid operator sends an OpenADR curtailment signal, the aggregator translates it into targeted BACnet writes at precisely the addresses the ontology specifies, within the boundaries agreed contractually with the building owner. The BMS continues to run its reliability logic; the aggregator adjusts only the setpoints that were explicitly licensed under the agreement.

The new business modes. The ASHRAE 223P model can unlock new business modes for aggregators and building owners. For example, an aggregator can offer building owners the following value proposition: “We will create and maintain your building’s semantic model, provide you with monitoring dashboards and fault detection analytics powered by that model, and in exchange, we receive the right to adjust specific setpoints within agreed comfort and reliability bounds during grid events.” In this example, the building owner receives a service (the ontology, the analytics); the aggregator receives dispatchable capacity; the grid receives a standardized, scalable enrollment path.

3 SI-Mapper: An Agentic Approach to Building Semantic Modeling

The previous section established that manual BACnet point mapping is costly, labor-intensive, and resistant to automation with existing tools. This section introduces the proposed solution of this report, agentic AI for automated HVAC semantic modeling, and traces its full development from initial concept through nine experimental iterations to the final architecture.

3.1 Overview of the Approach

SI-Mapper takes raw building data inputs, like HVAC engineering drawings (PNG or PDF) and BACnet point exports (CSV files), and produces an ASHRAE 223P semantic model in Turtle (TTL) format. The entire pipeline is orchestrated by an AI agent application guided by domain-specific skill instructions. Figure 1 illustrates the data flow.

The key enabler is multimodal AI. Recent large language models like Gemini 3.1 and Claude Sonnet 4.6 can analyze engineering drawings directly, identifying equipment symbols, ductwork connections, and spatial relationships from visual inputs. HVAC schematic diagrams, with their regular symbol vocabulary and consistent layout conventions, are more tractable for AI recognition than general architectural floor plans. The insight from SI-Mapper’s development is that standard models underperform on domain-specific drawing recognition until guided by detailed, domain-specific instructions, the *skills* architecture described in Section 3.4.

SI-Mapper’s novelty lies in combining four capabilities that no prior system integrates:

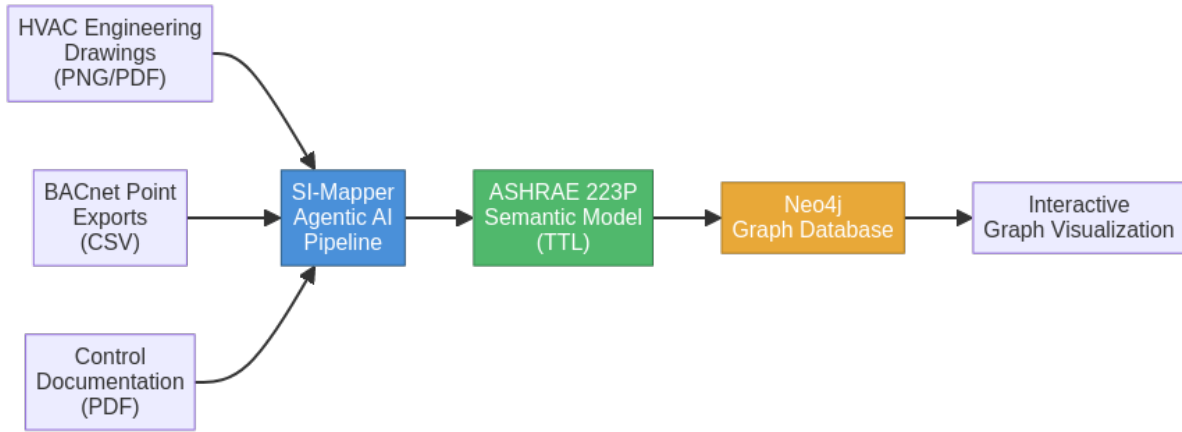


Figure 1: SI-Mapper data pipeline — from raw building data to semantic model and graph visualization.

1. **Multimodal drawing recognition:** AI analysis of HVAC engineering drawings to extract equipment and ductwork topology.
2. **BACnet mapping:** automated assignment of BACnet point data to the equipment identified in the drawings.
3. **ASHRAE 223P generation:** production of a formally valid ASHRAE 223P ontology including ConnectionPoints, Connections, mediums, and telemetry links.
4. **Iterative self-correction:** the agent compares its generated model against validation rules, identifies errors, and corrects them autonomously — looping until the model is valid.

This approach replaces weeks of expert engineering effort with an automated pipeline that runs in minutes, producing a machine-readable semantic model that unlocks software portability, advanced analytics, and grid integration for the building.

3.2 Development Methodology: Nine Experimental Iterations

The development of SI-Mapper followed an iterative experimental methodology. Over nine distinct iterations, the system architecture evolved from simple function-calling scripts to a sophisticated skills-based agentic AI system. Each iteration was driven by the failures and limitations of the previous one.

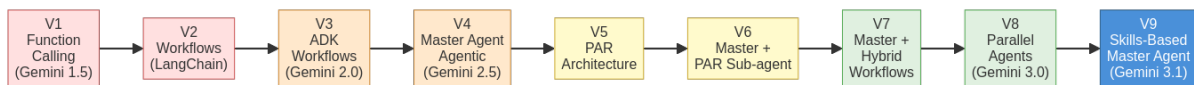


Figure 2: Evolution of SI-Mapper architecture across nine experimental iterations.

Iteration 1: LLMs as Function Providers (Gemini 1.5 Flash). The first approach used LLMs as providers of intelligence embedded inside conventional Python functions: the model was called for specific tasks such as image recognition or text generation, while all business logic remained hardcoded by the developer. This approach was straightforward but fundamentally limited, the LLM had no awareness of the application context and could not make autonomous decisions. Every new capability required writing new code around it. LangChain was used as the integration library at this stage.

Iteration 2: Workflow-Based Agents (Gemini 1.5 Flash). The second iteration introduced workflows to define agent behavior as a sequence of ordered steps, making it possible to better exploit the LLM’s capabilities. However, the business logic was still hardcoded inside

the workflow definitions, and the agent remained unable to adapt to unexpected situations. LangGraph was evaluated at this stage but was abandoned.

Iteration 3: ADK Workflows (Gemini 2.0 Flash). The third iteration migrated to Google’s Agent Development Kit (ADK) while retaining the workflow-based execution model. A local RAG system was introduced to provide agents with on-demand access to documentation for HVAC equipment classification. ADK had been very recently released at this point, with limited documentation and community examples, meaning AI coding assistants could not reliably help with the library, a concrete limitation that slowed development velocity.

Iteration 4: Agentic Master Agent (Gemini 2.5 Flash). The fourth iteration represented the first agentic attempt: a single master agent given full autonomy to plan and execute all tasks. This introduced genuine decision-making capability but exposed a critical failure mode, the prompts required to express the full HVAC pipeline were too long and complex for the model to follow reliably. A simple while loop was not sufficient to keep the agent on track. This iteration also introduced the CopilotKit frontend, the Agent-to-UI protocol for real-time agent observability, and MCP tools for external tool management.

Iteration 5: Plan-Act-Review Architecture (Gemini 2.5 Flash). Inspired by established multi-agent design patterns, the fifth iteration decomposed execution into three sequential sub-agents: Planner, Actor, and Reviewer. This improved output quality but the agent consistently terminated tasks before completing the full pipeline. Adding a loop agent on top of the PAR structure improved results further but was still insufficient, the agent could not reliably complete all stages in a single run.

Iteration 6: Master Agent with PAR Sub-Agent (Gemini 2.5 Flash). The sixth iteration added a master agent that could choose to act directly or delegate to the PAR structure as a sub-agent. Both the master and the sub-agent were given full autonomy. Despite this added flexibility, the PAR sub-agent still could not produce results of sufficient quality. Frontend tooling was extended with tool call and state variable tracking to provide better visibility into the agent’s reasoning.

Iteration 7: Master Agent with Hybrid Workflows (Gemini 2.5 Flash). The seventh iteration replaced the PAR agent with a hybrid approach: specialized image recognition workflows for the drawing analysis stage, combined with an agentic planner for task orchestration. This created an architecture that was neither fully workflow-based nor fully agentic. The recognition quality was not good enough, and the architectural hybrid added complexity without proportional benefit. The entire drawing sequence was ultimately converted back to a workflow, but results remained insufficient.

Iteration 8: Parallel Agents (Gemini 2.5 Flash, then Gemini 3.0 Flash). The eighth iteration introduced parallel execution: one specialized agent per equipment type running simultaneously, replacing the sequential recognition agents. This significantly improved processing speed. A model upgrade to Gemini 3.0 Flash delivered a meaningful accuracy improvement. Gemini 3.0 Pro was also tested and produced higher quality results but required considerably more time.

Iteration 9: Skills-Based Master Agent (Gemini 3.1 / Claude Sonnet 4.6). The ninth and final iteration converted all sub-agents into skills, specialized instruction sets executed inline within the master agent’s context. This architectural simplification was made possible by the dramatically improved instruction-following capability of the latest models. The agent was also given the ability to visit the frontend via headless browser, capture a screenshot of the HVAC canvas, and compare it against the source drawing for visual self-correction. The architecture became dramatically simpler, easier to maintain, and produced better results than any previous iteration. Additionally, two subagents executed in sequence were maintained just to create the Python code and the TTL code of the graph. The reason to keep them as sub-agents instead of skills is because of the complexity of the task that they need to execute. Isolating the context for the execution of the creation of the code in sub-agents was the best alternative.

The key insight from this progression is that advances in foundation model capability made simpler architectures viable. The skills-based approach became possible only when models like Gemini 3.1 and Claude Sonnet 4.6 reached sufficient instruction-following capability to execute complex, multi-step domain tasks within a single agent context.

3.3 Final Architecture

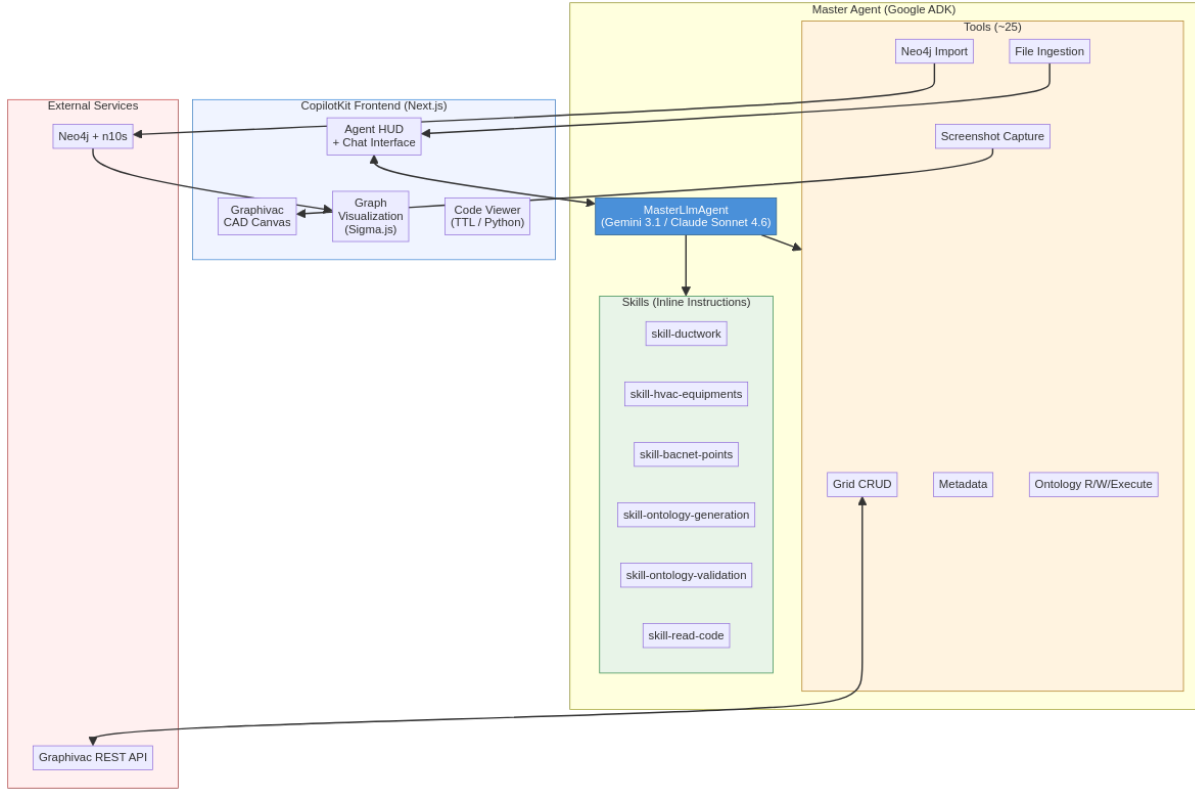


Figure 3: SI-Mapper final architecture — skills-based single master agent.

The final architecture organizes SI-Mapper into three main layers that together implement the full pipeline from raw Multi-modal data to a validated ASHRAE 223P semantic model stored in a graph database.

Frontend (CopilotKit + Next.js). The frontend serves as a real-time operations dashboard. A CopilotKit Agent acts as the chat interface. The middleware in ADK (before and after model callbacks) are used to stream the agent’s thoughts, actions and state modifications as they occur into the front end, giving operators full visibility into what the agent is doing and why. A Graphivac canvas renders the internal HVAC grid in real time as the agent places ducts and equipment using its tools. A code viewer shows the generated ASHRAE 223P Python code and the resulting TTL file. A Sigma.js graph viewer displays the final semantic model after it is loaded into Neo4j.

Master Agent (Google ADK). The core of the system is a master *LlmAgent* wrapped inside a loop Agent, built on Google’s Agent Development Kit, running on Gemini 3.1 or Claude Sonnet 4.6. The agent has access to a sequential sub-agent, approximately 25 tools and six skills. It maintains full context while extracting the data, drawing the HVAC system and BACnet metadata mapping. The master agent delegates the creation of the Python code and the generation of the TTL for the ontology to the sequential sub-agent.

External Services. Two external services underpin the system: Neo4j and Graphivac. Neo4j with the Neosemantics (n10s) plugin stores the ASHRAE 223P ontology as a native RDF

graph and provides Cypher query access for graph traversal. The Graphivac REST API manages the CAD canvas, allowing the agent to read and write the visual grid state bidirectionally.

The choice of a single-agent architecture with a single sequential sub-agent was driven by several compounding advantages. Skills provide domain expertise inline, making the agent’s behavior fully observable in a single context window without the opacity of delegated sub-agent calls. The agent naturally maintains full context across all pipeline stages, which is critical for operations such as visual self-correction, where the agent must compare a live screenshot of the canvas against the source drawing and determine what to fix. Finally, the architecture is dramatically simpler to maintain, debug, and extend: adding a new capability means writing a new skill or tool, not redesigning an agent coordination protocol.

3.4 Skills

Skills are specialized instruction sets that the master agent or the sequential agent can activate as needed. They are not separate agents, they act like a dynamic prompt. Each skill provides domain-specific knowledge for one stage of the pipeline, encoding expert-level HVAC drawing reading conventions and ontology generation rules that a general purpose language model would not apply consistently on its own.

Table 5: Master agent skills and their roles in the pipeline.

Skill	Purpose
skill-ductwork	Identify and place all ductwork from HVAC drawings as the spatial foundation
skill-hvac-equipments	Place HVAC equipment (fans, coils, dampers, sensors) onto the established duct s
skill-bacnet-points	Extract BACnet point data from CSV exports and map points to grid equipment
skill-ontology-generation	Generate ASHRAE 223P Python code from the populated grid state
skill-ontology-validation	Iteratively fix and validate ontology code to produce a clean TTL file
skill-read-code	Load error-resolution lessons from past coding sessions to avoid repeating mistakes

The skills follow a strict dependency chain that mirrors the physical reality of HVAC commissioning. Ductwork is placed first: all equipment must be anchored to a duct segment, so the duct grid is the spatial foundation on which everything else is built. Equipment placement follows, positioning fans, coils, dampers, and sensors onto the duct coordinates established in the previous step. BACnet mapping then attaches live point metadata to each piece of equipment on the grid. Each of these three stages concludes with a human verification step, allowing the operator to review and correct the agent’s work before the pipeline advances. Ontology generation reads the fully verified grid and produces Python code that models the system in ASHRAE 223P. Finally, ontology validation enters an iterative self-correction loop, executing and fixing the generated code until it produces a clean TTL file with no runtime errors. Each skill assumes the previous one has completed successfully; the master agent is responsible for detecting failures and deciding whether to retry or escalate.

3.5 Tools

The master agent has access to approximately 25 tools organized into ten categories. Tools handle all interactions with external systems, data storage, and the agent’s own operational state, the skills provide the reasoning; the tools provide the execution.

3.6 Key Technology Decisions

Several technology choices were made deliberately based on lessons from the iterative development process. Each decision reflects a tradeoff between capability, cost, and integration complexity.

Table 6: Master agent tools grouped by category.

Category	Tools	Description
Grid Sync	<code>sync_graphivac_to_agent</code> , <code>sync_agent_to_graphivac</code>	Bidirectional sync between agent state and Graphivac canvas
Grid CRUD	<code>add_component</code> , <code>add_components_batch</code> , <code>delete_component</code> , <code>delete_components_batch</code> , <code>read_internal_grid</code>	Create, read, and delete components in the internal grid
Metadata	<code>write_metadata</code> , <code>write_metadata_batch</code>	Attach BACnet point data to grid equipment
Ontology	<code>search_class_mapping</code> , <code>scan_python_files_filtered</code> , <code>read_ontology</code> , <code>write_ontology</code> , <code>execute_ontology</code> , <code>read_prompt</code>	ASHRAE 223P library lookup, code read/write/execute
Ontology Signals	<code>checkpoint_code</code> , <code>exit_generator_success</code> , <code>exit_generator_failure</code> , <code>exit_validator_success</code> , <code>exit_validator_failure</code>	Snapshot versioning and ontology task completion signaling
Neo4j	<code>load_ttl_to_neo4j</code>	Import validated TTL semantic model into Neo4j graph database
Frontend	<code>capture_frontend_state</code>	Screenshot the live canvas for visual self-correction
Ingestion	<code>ingest_category_files</code>	Load user-uploaded files (drawings, CSVs, PDFs) into session
State / Progress	<code>save_agent_state</code> , <code>get_agent_state</code> , <code>update_step</code> , <code>update_status</code>	Track processing progress and persist state across turns
Loop Control	<code>exit_loop_level_2</code>	Terminate the master agent session

Google ADK. Google’s Agent Development Kit was chosen as the agent runtime for its native Gemini integration and its built-in support for agent lifecycle management, tool registration, session state, and artifact handling. Earlier iterations used LangChain, but the tests were carried out at the early stages of the development of LangChain, and at the time the library still had many problems. ADK provided since its inception the necessary infrastructure to make the required tests. However, at the time of writing this report, new alternatives appear and others improve. Thus, at this time, considering the usage of LangChain, it will be comparable alternative to ADK. Moreover, the introduction of deep agents in LangChain might be even a better alternative. Additionally, testing new capabilities inside of the Claude code framework is something that must be executed in the future to have an overall picture of what could be the best choice for the agent framework for continuing the development.

CopilotKit. CopilotKit implements the Agent-to-UI protocol, enabling the Python ADK backend to stream agent thoughts and actions to the Next.js frontend. Without this library, building the chat interface would have required extra development with no research value. CopilotKit handles this concern, allowing development to focus on the agent pipeline itself. Nevertheless, its usage could be questioned for future iterations. Especially if future iterations focus on developing on top of Claude code, in which having a user interface could be built in a completely different manner.

Neo4j with Neosemantics (n10s). ASHRAE 223P is an RDF ontology; its natural storage format is a graph database with native RDF import capabilities. Neo4j with the Neosemantics (n10s) plugin satisfies both requirements: it stores the TTL file as a native graph, and it provides Cypher, a mature, expressive query language, for graph traversal and analysis. Alternative approaches such as storing TTL as flat files or using a triple store were considered, but Neo4j’s combination of graph-native storage and strong ecosystem tooling made it the pragmatic choice. Neo4j has a disadvantage in that it requires converting the TTL into an ATTIC graph, and some information might be lost in that conversion. The information lost was not analyzed in this iteration due to limitations in time. However, it is important to analyze it in the future to have a better understanding of the implications of information lost. GraphDB does not have the same problem because it doesn’t require a conversion from the TTL. However, GraphDB works using SPARQL instead of Cypher. Recent works have demonstrated that LLMs achieve higher accuracy when writing Cypher than when writing SPARQL queries [18].

Sigma.js for Graph Visualization. Sigma.js renders large graphs using WebGL, providing smooth performance on semantic models with hundreds of nodes and relationships. Heavier alternatives such as D3 or Gephi were considered, but both introduce unnecessary complexity for a browser-native visualization requirement. Sigma.js integrates cleanly with the Next.js frontend and requires no additional server-side rendering accommodations beyond a standard dynamic import guard.

4 Experiment description, analysis, and results

4.1 Experimental Setup

Two HVAC systems were selected for evaluation: System 1 (Air Handling Unit) and System 3 (Air Handling Unit). Both systems are drawn from real buildings with existing BACnet point exports and engineering drawings available for ingestion. Both systems are installed on a real building site, actual drawings of air handling units with real-world HVAC documentation, not synthetic test cases.

Four metrics are recorded for each AI agent run: total LLM token consumption (cumulative context window size across all model calls in the session), estimated token cost in USD at the applicable model pricing, wall-clock time from session start to validated TTL output, and a qualitative assessment of the generated ASHRAE 223P graph based on completeness

(all equipment and sensors present), accuracy (correct topology and BACnet references), and interpretability (graph readable by downstream applications).

The model used for both experiments is **Gemini 3.1 Pro Preview**, billed at \$2.00/M tokens for prompts up to 200 000 tokens and \$12.00/M for output and thinking tokens (same prompt size tier).

4.2 System 1: Air Handling Unit

4.2.1 Input Materials

System 1 is a real-world Air Handling Unit (AHU) from an existing building installation. The agent was provided three input artifacts: an HVAC engineering schematic drawing (PNG), a BACnet point export CSV file containing raw control point data, and a control documentation PDF. No additional context was given.

Figure 4 shows the original engineering drawing alongside the AI-generated grid replication produced by the agent before ontology generation.

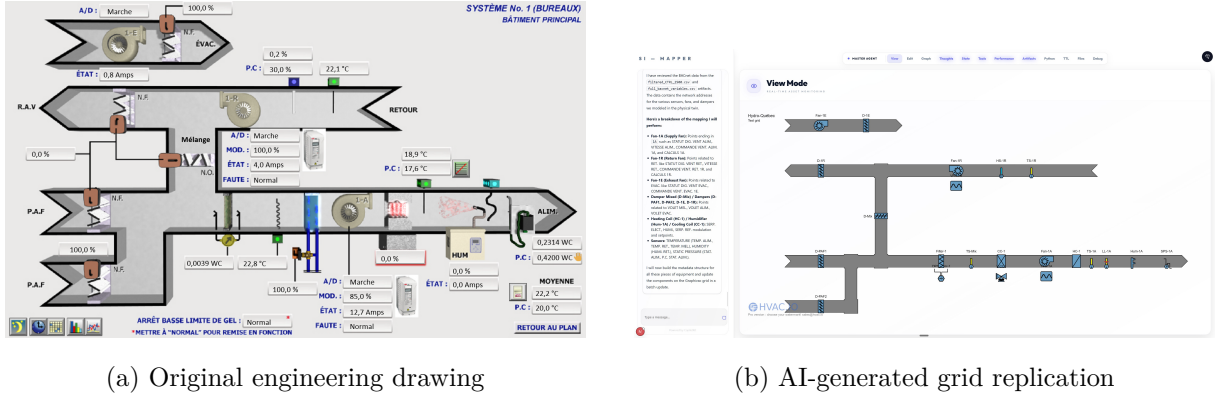


Figure 4: System 1 — original HVAC schematic (left) and the agent’s replication on the Graphivac grid (right), produced as an intermediate step before ASHRAE 223P generation.

4.2.2 Performance Metrics

Table 7: System 1 (AHU) — AI Agent Performance

Metric	Value
Model used	Gemini 3.1 Pro Preview
Wall-clock time	17 min 11 s
Peak context window	185,735 tokens
Estimated token cost	\$11.30 USD
HVAC components placed	21 (3 fans, 2 coils, 5 dampers, 6 sensors, 1 filter, 1 humidifier, 2 VFDs, 1 valve)
BACnet points mapped	44 points → 17 equipment instances
Validation iterations	3 (auto-corrected by the agent)
ASHRAE 223P graph	295 nodes, 949 relationships
Session outcome	Success

4.2.3 Graph Quality Assessment

The agent successfully identified and placed all 21 HVAC components from the engineering drawing, including fans, dampers, coils, and every instrumented sensor. The 44 BACnet data points from the controls CSV were extracted and correctly associated with 17 equipment instances, covering supply/return temperatures, damper positions, fan speeds, static pressure, and humidity. Three minor code errors surfaced during ontology validation — incorrect import names, a missing constructor parameter, and a Python operator syntax issue — all of which the agent detected and corrected autonomously within the same session, requiring no human intervention.

The resulting ASHRAE 223P graph contains 295 nodes and 949 relationships, encoding the full equipment topology (inlet and outlet connection points, duct segments, medium types), all BACnet external references, and the spatial composition of the AHU assembly. Post-generation Cypher queries confirmed that fans, dampers, and sensor-to-BACnet linkages were all correctly represented in the Neo4j database.

Figure 5 shows the generated ASHRAE 223P semantic graph as visualized in the SI-Mapper frontend.

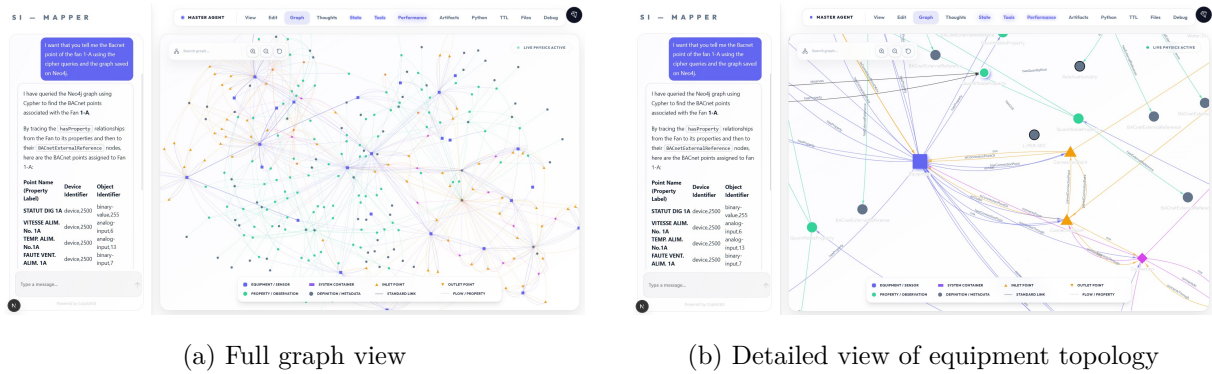


Figure 5: System 1 — generated ASHRAE 223P semantic graph in the SI-Mapper Neo4j visualization (295 nodes, 949 relationships). The detailed view shows equipment nodes with their connection points and BACnet external references.

4.3 System 3: Air Handling Unit

Table 8: System 3 (AHU) — AI Agent Performance

Metric	Value
Model used	[TODO: Fill after running experiment]
Wall-clock time	[TODO: Fill after running experiment]
Peak context window	[TODO: Fill after running experiment]
Estimated token cost	[TODO: Fill after running experiment]
HVAC components placed	[TODO: Fill after running experiment]
BACnet points mapped	[TODO: Fill after running experiment]
Validation iterations	[TODO: Fill after running experiment]
ASHRAE 223P graph	[TODO: Fill after running experiment]
Session outcome	[TODO: Fill after running experiment]

[TODO: Insert graph visualization screenshots for System 3 (full view and detail view).]

[TODO: Write qualitative graph quality assessment for System 3 once experiment is complete.]

4.4 Human Engineer Baseline

For comparison, an experienced HVAC engineer will perform the same task manually: reading the same engineering drawings and BACnet point export CSV files, and producing an equivalent ASHRAE 223P semantic model by hand using domain knowledge and available ontology tooling. The human engineer is given the same inputs as the AI agent — no additional context — to ensure a fair comparison of effort and output quality.

Table 9: Human Engineer Baseline

Metric	System 1	System 3
Time to complete	[TODO: Fill after human trial]	[TODO: Fill after human trial]
Estimated labor cost (USD)	[TODO: Fill after human trial]	[TODO: Fill after human trial]
ASHRAE 223P graph quality	[TODO: Fill after human trial]	[TODO: Fill after human trial]

4.5 Comparison: AI Agent vs. Human Engineer

Table 10: AI Agent vs. Human Engineer — System 1 and System 3

	AI Agent	Human Engineer	Ratio
System 1 — Time	17 min 11 s	[TODO: Human time]	[TODO: Nx faster]
System 1 — Cost	\$11.30 USD	[TODO: Human cost]	[TODO: Nx cheaper]
System 3 — Time	[TODO: AI time]	[TODO: Human time]	[TODO: Nx faster]
System 3 — Cost	[TODO: AI cost]	[TODO: Human cost]	[TODO: Nx cheaper]

[TODO: Write comparative analysis once both System 3 AI experiment and human baseline trials are complete. Discuss time and cost ratios, graph quality comparison (completeness and accuracy of BACnet mappings), cases where the AI struggled (e.g. ambiguous drawing symbols), and implications for production deployment at scale.]

5 Conclusions

5.1 Summary of Contributions

This report addressed a documented, economically significant problem in the building industry: manual BACnet point mapping costs 0.5–1.5% of construction value per building, blocks software interoperability, and is identified in peer-reviewed literature and DOE policy reports as a major impediment to building automation innovation. The same problem directly prevents the large-scale enrollment of commercial buildings into Virtual Power Plant and Non-Wire Alternative programs, because each building requires a custom, per-project integration effort estimated at \$5,000–\$15,000 before it can participate in grid services. No widely-deployed automated solution for this problem existed prior to this work.

SI-Mapper demonstrates that agentic AI can automate the creation of ASHRAE 223P semantic building models from two raw inputs available on any real building project: HVAC engineering drawings and BACnet point export CSV files. The system combines multimodal drawing recognition — using foundation models capable of interpreting HVAC schematic symbols directly from images — with automated BACnet point-to-equipment mapping and formal ASHRAE 223P ontology generation in a single, end-to-end pipeline. The agent validates its own output through iterative self-correction, looping until the generated Turtle (TTL) file executes

without errors and the resulting graph loads cleanly into the Neo4j semantic store. This pipeline replaces a workflow that previously required weeks of expert HVAC engineering labor.

The development process itself is a contribution. Nine experimental iterations, from simple LangChain function-calling in 2024 to the final skills-based architecture on Gemini 3.1 and Claude Sonnet 4.6 in 2026, constitute a systematic record of how agentic AI architectures for domain-specific technical tasks evolved as foundation model capabilities improved. The progression reveals a counterintuitive architectural lesson: simpler architectures became more viable, not less, as models improved. Multi-agent designs with dedicated Planner, Actor, and Reviewer sub-agents — initially necessary to keep the pipeline on track — became unnecessary once models reached sufficient instruction-following capability. The final skills-based single master agent outperformed every multi-agent architecture tried while being dramatically simpler to maintain, debug, and extend.

The technology stack assembled for SI-Mapper — Google ADK, CopilotKit, Neo4j with Neosemantics (n10s), and Sigma.js — provides a reusable reference architecture for agentic AI applications requiring real-time operator observability, semantic graph storage, and interactive graph visualization. The combination of ADK for agent orchestration, CopilotKit for the Agent-to-UI streaming protocol, and Neo4j for native RDF graph storage is not an obvious combination; the design decisions that led to it are documented in Section 3 as a practical guide for future implementors.

5.2 Limitations

Model dependency. The system’s performance is directly coupled to the capabilities and availability of specific foundation models — Gemini 3.1 and Claude Sonnet 4.6 as of this writing. Model API changes, pricing changes, or capability regressions in future model versions could affect pipeline reliability without any change to the SI-Mapper codebase. The skills architecture mitigates this risk somewhat by encapsulating domain knowledge in plain-text instruction sets that can be updated independently of model versions, but the underlying dependency on external model providers remains.

Limited validation scope. **[TODO: Update after experiments — describe how many systems were tested (System 1 and System 2), overall accuracy rates, and any cases where the agent failed to produce a valid ASHRAE 223P model on the first or second run.]** The current experimental scope covers a small number of real HVAC systems drawn from a single project context. Statistical confidence in accuracy and generalizability requires a larger validation corpus.

Domain specificity. The current skills and tool set are tailored to HVAC air handling units with BACnet points in French-language naming conventions, reflecting the Quebec installation context where this work was developed. Extending SI-Mapper to other building system types (VAV boxes, chiller plants, boiler plants), other control protocols (Modbus, KNX, BACnet/IP vs. BACnet MSTP), or BACnet point naming conventions from other regions and contractors would require additional skill development and validation. The architecture supports this extension naturally — new skills are plain-text instruction sets — but the domain knowledge required to write accurate skills is non-trivial.

ASHRAE 223P maturity. The standard is currently in its second advisory public review and is not yet ratified. The ontology generation logic in SI-Mapper is calibrated against the current advisory draft. Changes to the standard during ratification — particularly to connection point types, medium definitions, or telemetry binding syntax — could require updates to the ontology generation skills and the underlying ASHRAE 223P Python library used to produce the TTL output.

5.3 Future Work

Production validation at scale. The most important next step is running SI-Mapper against a larger corpus of real buildings — a target of 10–20 HVAC systems spanning multiple system types, contractors, and naming conventions. This would establish statistical confidence in accuracy and cost savings, identify edge cases not encountered in the current development set, and produce the benchmark data needed for peer-reviewed publication.

Multi-system support. The current skills cover air handling units. A production-viable system would need skills for VAV terminal boxes, chiller plants, boiler plants, heat pump systems, and mechanical ventilation units. Each system type has its own drawing conventions, point naming patterns, and ASHRAE 223P topology rules. The skills architecture makes this extension tractable: each system type requires a new skill file rather than a new agent or pipeline redesign.

VPP integration pilot. The highest-impact near-term application of SI-Mapper output is a demonstration connecting the generated ASHRAE 223P model to a VPP enrollment platform. Such a pilot would demonstrate the complete value chain — from raw HVAC drawings to grid dispatch-ready semantic model — and validate the hypothesis that semantic models are the interoperability layer that eliminates the per-building custom integration step currently blocking VPP scale.

Automated quality metrics. The current evaluation of generated ASHRAE 223P models is qualitative. A production evaluation framework would include automated comparison tools: graph diffing against a human-authored ground-truth model, quantitative scores for completeness (fraction of expected equipment nodes present), accuracy (fraction of BACnet references correctly assigned), and topology correctness (fraction of directed connections matching the source drawing). These metrics would make it possible to track regression across model updates and skill revisions without manual inspection.

Multi-language support. Adapting the BACnet point mapping skills to recognize English, Spanish, and other naming conventions beyond French would expand SI-Mapper’s applicable geography significantly. The mapping skills are the primary location where naming convention knowledge is encoded; adding language variants is a skill-writing task rather than an architectural change.

References

- [1] Kim Trenbath, Ryan Meyer, Korbaga Woldekidan, Kristi Maisha, and Morgan Harris. Commercial building sensors and controls systems: Barriers, Drivers, and Costs. Technical Report NREL/TP-6A50-82117, National Renewable Energy Laboratory, Golden, CO, August 2022. URL <https://docs.nrel.gov/docs/fy22osti/82117.pdf>. Prepared for the U.S. Department of Energy Building Technologies Office.
- [2] Building Commissioning Association. Building commissioning: Costs and Benefits (industry guidance). Industry guidance document, 2024. URL <https://www.bcxa.org>.
- [3] PingCx. The hidden costs of manual commissioning in today’s complex buildings. Blog post, 2024. URL <https://www.pingcx.com/blog/the-hidden-costs-of-manual-commissioning-in-todays-complex-buildings>.
- [4] Envigilance. Building energy monitoring cost. Blog post, 2023. URL <https://envigilance.com/blog/building-energy-monitoring-cost/>.
- [5] Access Coins. Beat the HVAC technician shortage. Industry article, republished by SMACNA, 2025. URL <https://www.smacna.org/news/products-and-services-updates/article/2025/10/09/beat-the-hvac-technician-shortage>.

- [6] Weimin Wang, Michael R. Brambley, Woohyun Kim, Sriram Somasundaram, and Andrew J. Stevens. Automated point mapping for building control systems: Recent advances and future research needs. *Automation in Construction*, 84:107–124, 2017. doi: 10.1016/j.autcon.2017.09.013. Available via OSTI ID 1495345.
- [7] Marco Perini, Daniele Antonucci, and Rocco Giudice. BrickLLM: A Python library for generating Brick-compliant RDF graphs using LLMs. *SoftwareX*, 2025. doi: 10.1016/j.softx.2025.102083. URL <https://www.sciencedirect.com/science/article/pii/S2352711025000883>.
- [8] National Renewable Energy Laboratory (NREL), NIST, LBNL, and PNNL. BuildingMOTIF: Building metadata OnTology interoperability framework. Open-source software (GitHub), 2023. URL <https://github.com/NREL/BuildingMOTIF>.
- [9] Open223 Project Contributors. Open223 ASHRAE 223p reference models. Open-source model repository, 2025. URL <https://models.open223.info/>.
- [10] Willow Technology Corporation. Willow AI-driven digital twin platform. Company website and press release (AI Breakthrough Awards, June 2025), 2025. URL <https://willowinc.com/willow-named-ai-startup-of-the-year-in-8th-annual-ai-breakthrough-awards-program/2548>.
- [11] SkyFoundry. SkySpark analytics platform. Product website, 2024. URL <https://skyfoundry.com/product>.
- [12] U.S. Department of Energy. Semantic modeling and interoperability. DOE Buildings program page, 2024. URL <https://www.energy.gov/eere/buildings/semantic-modeling-and-interoperability>.
- [13] Wood Mackenzie. 2025 North America virtual power plant market report. Technical report, Wood Mackenzie, September 2025. URL <https://www.woodmac.com/press-releases/virtual-power-plant-capacity-expands-13.7-year-over-year-to-reach-37.5-gw>.
- [14] RMI (Rocky Mountain Institute). VP3 progress report: From awareness to action. Technical report, Rocky Mountain Institute / Virtual Power Plant Partnership, April 2025. URL <https://rmi.org/insight/vp3-progress-report-from-awareness-to-action/>.
- [15] Smart Electric Power Alliance (SEPA) and NC Clean Energy Technology Center. 50 states of virtual power plants and supporting distributed energy resources: 2024 state policy snapshot. Technical report, SEPA / NC Clean Energy Technology Center, February 2025. URL <https://sepapower.org/resource/50-states-of-virtual-power-plant-and-supporting-distributed-energy-resources-2024-state>
- [16] California Energy Commission. Virtual power plant approaches for demand flexibility (VPP-FLEX), grant GFO-23-309. Grant award announcement, 2024. URL <https://www.energy.ca.gov/solicitations/2024-03/gfo-23-309-virtual-power-plant-approaches-demand-flexibility-vpp-flex>.
- [17] U.S. Department of Energy. Pathways to commercial liftoff: Virtual Power Plants 2025 update. Technical report, U.S. Department of Energy, Office of Technology Transitions, January 2025. URL https://climateprogramportal.org/wp-content/uploads/2025/06/LIFTOFF_DOE_VPP_2025_Update.pdf.
- [18] Sithursan Sivasubramaniam, Cedric Osei-Akoto, Yi Zhang, Kurt Stockinger, and Jonathan Fuerst. SM3-Text-to-Query: Synthetic multi-model medical text-to-query benchmark. *arXiv*

preprint arXiv:2411.05521, November 2024. URL <https://arxiv.org/abs/2411.05521>.
Benchmarks LLM text-to-query performance across Cypher, SPARQL, and SQL; finds LLMs achieve higher accuracy on Cypher than SPARQL.